TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐHQG-HCM
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Môn học: Nền tảng lập trình cho phân tích và trực quan dữ liệu (CO5177)
Học kỳ 261, năm học 2026–2027

BÁO CÁO BÀI TẬP LỚN

Phân tích dữ liệu dạng bảng

Dữ liệu bảng (Tabular)

Nhóm: DataInsight
Giảng viên: Lê Thành Sách
Landing page: https://trthminh.github.io/CO5177/

Họ tên	MSSV	Phụ trách
Trương Thanh Minh	2580893	Chuỗi thời gian (Time series)
Tô Anh Phát	2580894	Văn bản (Text)
Trần Lê Hoàng Long	2570443	Dữ liệu bảng (Tabular)

TP. Hồ Chí Minh, tháng 11 năm 2026

Mục lục

1 Phân tích dữ liệu dạng bảng **1**

1.1 Bài toán và mục tiêu . . . 1
1.2 Tập dữ liệu . . . 1
1.3 Kiểm tra chất lượng dữ liệu . . . 2
1.4 Phân tích khám phá (EDA) . . . 3
1.4.1 Câu hỏi 1: Đặt phòng càng sớm thì càng dễ hủy? . . . 3
1.4.2 Câu hỏi 2: Vì sao booking Non Refund lại bị hủy nhiều nhất? . . . 3
1.4.3 Câu hỏi 3: Yêu cầu đặc biệt và khách quen có làm giảm hủy phòng? . . . 4
1.5 Tiền xử lý dữ liệu . . . 5
1.5.1 Pipeline và chống rò rỉ dữ liệu . . . 5
1.5.2 Mã hóa và chuẩn hóa . . . 5
1.5.3 So sánh các cách xử lý giá trị thiếu . . . 5
1.5.4 Xử lý outlier . . . 6
1.6 Mô hình và thiết lập thí nghiệm . . . 6
1.6.1 Thí nghiệm rò rỉ dữ liệu . . . 6
1.6.2 Xử lý mất cân bằng lớp . . . 7
1.7 Kết quả và đánh giá . . . 7
1.8 Hình trả lời câu hỏi . . . 9
1.9 Kết luận . . . 10
1.10 Hạn chế và hướng phát triển . . . 10
1.11 Tài liệu tham khảo . . . 10

Chương 1

Phân tích dữ liệu dạng bảng

1.1 Bài toán và mục tiêu

Khi khách hủy phòng sát ngày đến, khách sạn thường không kịp bán lại phòng đó. Nếu biết trước booking nào dễ bị hủy, khách sạn có thể gọi điện xác nhận, yêu cầu đặt cọc hoặc nhận thêm một vài booking để bù vào. Chương này dự đoán một booking có bị hủy hay không (`is_canceled` = 1) chỉ dựa trên những thông tin có tại lúc khách đặt phòng, gồm số ngày đặt trước, số đêm, số khách, kênh bán, loại đặt cọc, giá phòng và lịch sử của khách.

Số booking bị hủy ít hơn hẳn số booking không hủy, nên độ chính xác (accuracy) không phản ánh đúng khả năng phát hiện booking hủy. Các mô hình vì vậy được so sánh chủ yếu bằng **PR-AUC** (average precision) và **F1 của lớp hủy** trên tập validation; accuracy chỉ để tham khảo. Mô hình được chọn chỉ được đánh giá một lần trên tập test.

1.2 Tập dữ liệu

- Tên: Hotel Booking Demand, bản TidyTuesday ngày 11/02/2020 (giống hệt bản trên Kaggle).
- Nguồn: `https://github.com/rfordatascience/tidytuesday/tree/master/data/2020/2020-02-11`; notebook tải trực tiếp file `hotels.csv` nên không cần API key.
- Thu thập: trích từ hệ thống quản lý khách sạn (PMS) của hai khách sạn ở Bồ Đào Nha, một resort ở vùng Algarve (H1, 40.060 booking) và một khách sạn trong thành phố Lisbon (H2, 79.330 booking), gồm mọi booking có ngày đến từ tháng 7/2015 đến tháng 8/2017, kể cả booking đã hủy. Thông tin nhận dạng khách hàng đã bị xóa.
- Kích thước: 119.390 dòng, 32 cột; mỗi dòng là một booking.
- Biến mục tiêu: `is_canceled`, 37,0% booking bị hủy trên dữ liệu gốc.
- Giá trị thiếu: 129.425 ô, nằm ở `company` (94,3%), `agent` (13,7%), `country` (0,4%) và `children` (4 ô).
- Giấy phép và trích dẫn: CC BY 4.0. N. António, A. de Almeida, L. Nunes, *Hotel booking demand datasets*, Data in Brief 22 (2019) 41–49.

Chọn cột theo thời điểm dự đoán. Từng cột được kiểm tra xem đã có tại lúc đặt phòng hay chỉ được ghi sau đó. Bốn cột chỉ có sau khi đặt nên bị loại: `reservation_status` và `reservation_status_date` (chính là kết quả hủy hay không và ngày ghi kết quả), `assigned_room_type` (loại phòng thực nhận lúc check-in) và `booking_changes` (số lần sửa booking cho đến lúc check-in hoặc hủy). `arrival_date_year` cũng bị loại vì chỉ gồm các năm 2015–2017, không dùng được cho năm sau. Còn lại 26 đặc trưng gồm 15 cột số và 11 cột phân loại; `agent` và `company` lưu dưới dạng số nhưng là mã ID của đại lý và công ty, nên được xếp vào nhóm phân loại.

1.3 Kiểm tra chất lượng dữ liệu

Bảng 1.1: Các vấn đề chất lượng dữ liệu và cách xử lý

Vấn đề	Quy mô	Cách xử lý
Thiếu `company`, `agent`	94,3% và 13,7% số dòng	Giữ thành một giá trị riêng `"missing"`, thêm hai cột cờ `has_company`, `has_agent`
Thiếu `country`, `children`	488 và 4 ô	Cùng cách trên; riêng `children` điền bằng median của tập train
Dòng trùng hoàn toàn	31.994 dòng (26,8%); 34.050 dòng khi bỏ các cột bị loại	Bỏ trước khi chia tập
`adr` âm hoặc quá lớn	1 dòng $-6{,}38$, 1 dòng 5.400	Bỏ (khoảng hợp lệ 0–1.000)
Booking không có khách	180 dòng	Bỏ bằng quy tắc trên nhiều cột
`meal = Undefined`	1.169 dòng	Gộp với `SC` (không có gói ăn)
Đuôi phải dài (`lead_time`, `adr`, số đêm)	tới 737 ngày, 50 đêm	Giữ nguyên; dùng `log1p` và `RobustScaler`

Giá trị thiếu. Việc thiếu dữ liệu ở đây không xảy ra ngẫu nhiên. Booking không có `company` bị hủy 38,2%, trong khi booking có `company` chỉ hủy 17,5%. Với `agent` thì ngược lại: không qua đại lý hủy 24,7%, có đại lý hủy 39,0%. Ô trống ở `agent` hay `company` thực chất có nghĩa là booking không đặt qua đại lý hoặc công ty nào, nên các ô này được giữ lại như một loại thông tin thay vì điền bằng giá trị khác.

Dòng trùng. Dataset không có mã booking, nên khi một đoàn đặt nhiều phòng giống nhau thì các phòng đó trở thành nhiều dòng giống hệt nhau. Các bản sao khác hẳn phần còn lại: 63,1% bị hủy (so với 27,5%), 42,3% là Non Refund (so với 1,2%) và 46,5% thuộc phân khúc Groups (so với 5,7%). Có tới 13.549 trong 14.587 booking Non Refund là bản sao. Nếu giữ lại, cùng một booking có thể nằm ở cả tập train và tập test, làm kết quả đánh giá cao hơn thực tế, nên các dòng trùng bị bỏ. Sau khi làm sạch còn 85.175 booking và tỷ lệ hủy giảm từ 37,0% xuống 27,7%.

Outlier. Quy tắc IQR không phù hợp với các cột đếm. Ví dụ ở `adults`, $Q_1 = Q_3 = 2$ nên mọi booking không phải 2 người lớn đều bị coi là outlier (24,9% số dòng). Các giá trị lớn

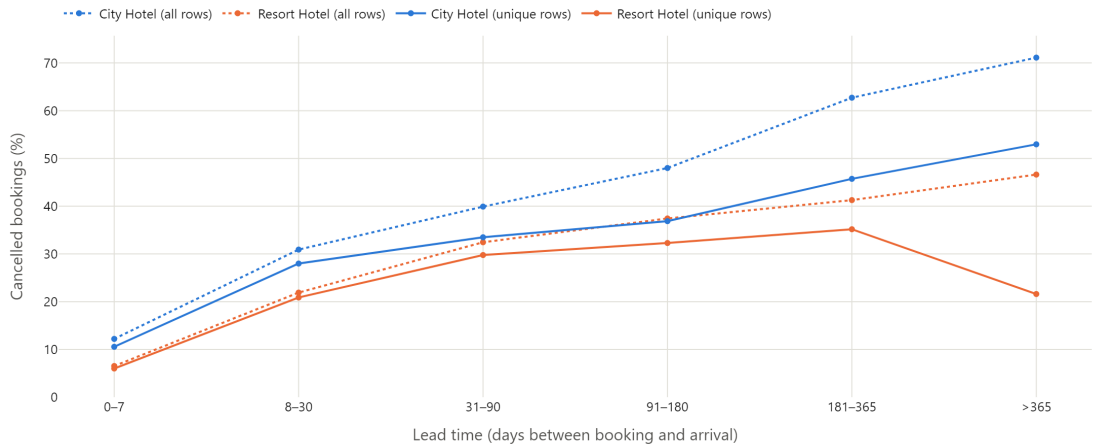
còn lại như booking đoàn 55 người lớn hay booking ở hơn 30 đêm vẫn có thể xảy ra, nên được giữ nguyên.

1.4 Phân tích khám phá (EDA)

Một số quy luật đã thấy rõ trước khi xây dựng mô hình. Trong 7.416 booking có yêu cầu chỗ đỗ xe, không có booking nào bị hủy. Khách Bồ Đào Nha hủy 56,6% số booking, gấp khoảng ba lần khách Đức, Pháp và Anh (16,7–20,2%), và City Hotel có tỷ lệ hủy cao hơn Resort Hotel (41,7% so với 27,8%). Vì dòng trùng tập trung ở một số nhóm, ba câu hỏi dưới đây được tính trên cả toàn bộ dữ liệu và dữ liệu đã bỏ trùng, tức dữ liệu dùng để huấn luyện mô hình về sau.

1.4.1 Câu hỏi 1: Đặt phòng càng sớm thì càng dễ hủy?

Trên dữ liệu đã bỏ trùng, tỷ lệ hủy của City Hotel tăng từ 10,5% với booking đặt trước tối đa 7 ngày lên 45,7% với booking đặt trước 181–365 ngày. Resort Hotel có cùng xu hướng trong khoảng một năm (từ 6,0% lên 35,2%). Dòng trùng làm các nhóm đặt sớm bị phóng đại: nếu tính trên toàn bộ dữ liệu, nhóm 181–365 ngày ở City Hotel lên tới 62,7%.



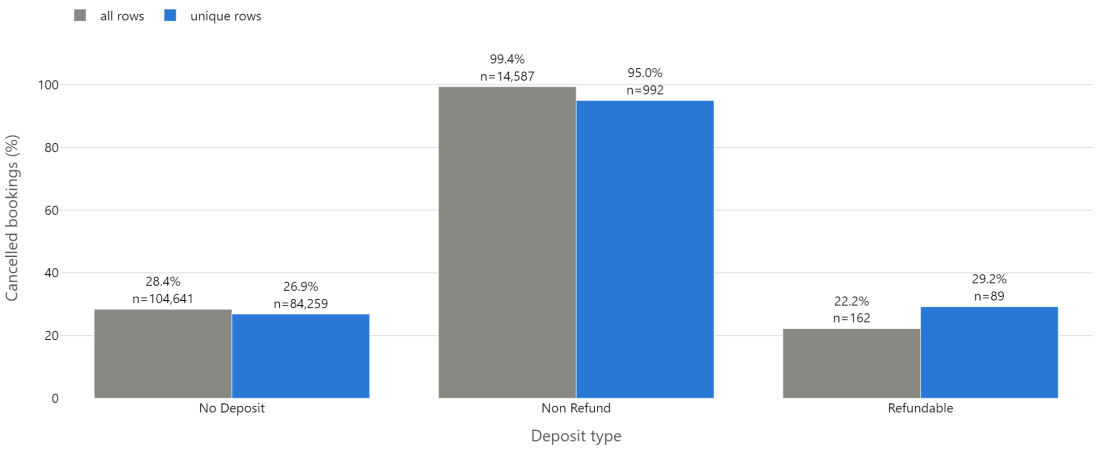
Hình 1.1: Tỷ lệ hủy tăng theo thời gian đặt trước: ở City Hotel là 10,5% khi đặt trước tối đa 7 ngày và 45,7% khi đặt trước 181–365 ngày (đường liền: dữ liệu đã bỏ trùng, đường chấm: toàn bộ dữ liệu).

Như vậy thời gian đặt trước liên hệ rõ với việc hủy phòng, nhưng quan hệ này không tuyến tính và khác nhau giữa hai khách sạn. Đây là lý do các mô hình dạng cây được kỳ vọng cho kết quả tốt hơn mô hình tuyến tính.

1.4.2 Câu hỏi 2: Vì sao booking Non Refund lại bị hủy nhiều nhất?

Thông thường khách đã trả tiền không hoàn lại sẽ ít hủy, nhưng trong dữ liệu booking Non Refund bị hủy 99,4%. Sau khi bỏ trùng tỷ lệ này vẫn là 95,0%, nhưng chỉ còn 992 booking. So với booking không đặt cọc, phần lớn là khách đoàn hoặc khách qua hãng lữ hành (62,8% Groups và 28,2% Offline TA/TO), gần như toàn bộ là khách Bồ Đào Nha (94,9%), và đặt trước trung vị 192 ngày thay vì 47 ngày. Có thể đây là các công ty lữ hành giữ phòng từ sớm rồi trả lại khi

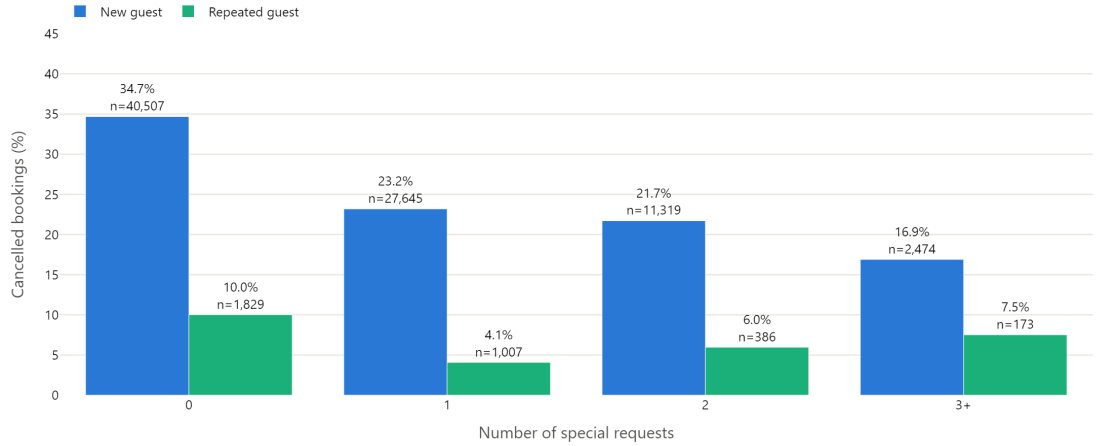
không bán hết, khi đó điều khoản không hoàn tiền nằm trong hợp đồng với đại lý chứ không phải khách lẻ đã thanh toán. Dữ liệu không có cột nào xác nhận trực tiếp giả thuyết này.



Hình 1.2: Booking Non Refund vẫn bị hủy 95,0% sau khi bỏ trùng, nhưng chỉ 992 trong 14.587 dòng Non Refund là không trùng.

1.4.3 Câu hỏi 3: Yêu cầu đặc biệt và khách quen có làm giảm hủy phòng?

Với khách mới, tỷ lệ hủy giảm từ 34,7% khi không có yêu cầu đặc biệt nào xuống 23,2% khi có một yêu cầu và 16,9% khi có từ ba yêu cầu trở lên. Khách quen chỉ hủy 7,7% so với 28,5% ở khách mới, và hủy ít hơn ở mọi mức số yêu cầu. Dòng trùng chỉ ảnh hưởng tới nhóm không có yêu cầu đặc biệt (47,7% trên toàn bộ dữ liệu so với 33,6% sau khi bỏ trùng), cho thấy các bản sao hầu hết là booking không có yêu cầu nào.



Hình 1.3: Chỉ một yêu cầu đặc biệt đã làm tỷ lệ hủy của khách mới giảm từ 34,7% xuống 23,2%; khách quen hủy 7,7% so với 28,5% ở khách mới.

1.5 Tiền xử lý dữ liệu

1.5.1 Pipeline và chống rò rỉ dữ liệu

Các quy tắc xử lý theo từng dòng (chuẩn hóa chữ, bỏ dòng trùng và giá trị vô lý, tạo `total_nights`, `total_guests`, `has_agent` và `has_company`) được thực hiện trước khi chia tập vì chúng không dùng thông tin của các dòng khác. Các thống kê như median để điền giá trị thiếu, tham số chuẩn hóa hay danh sách giá trị cho one-hot được học trong `Pipeline` với `ColumnTransformer`, nên chỉ lấy từ tập train, kể cả khi chạy cross-validation.

Bảng 1.2: Chia train / validation / test theo tỷ lệ 70/15/15, có phân tầng theo nhãn

Tập	Số dòng	Tỷ lệ hủy
Train	59.622	27,7%
Validation	12.776	27,7%
Test	12.777	27,7%

1.5.2 Mã hóa và chuẩn hóa

Cột số được điền bằng median của tập train và chuẩn hóa bằng `RobustScaler`; cách này dùng median và IQR nên ít bị ảnh hưởng bởi các giá trị lớn ở đuôi phân phối. Cột phân loại được mã hóa one-hot, những giá trị chiếm dưới 1% tập train được gộp thành một cột, nhờ đó `country`, `agent` và `company` không sinh ra hàng trăm cột. Tháng đến có thứ tự tự nhiên nên được mã hóa bằng `OrdinalEncoder` theo thứ tự tháng. Sau tiền xử lý có 81 cột đầu vào cho mô hình.

1.5.3 So sánh các cách xử lý giá trị thiếu

Bốn cách được so sánh trên tập validation, cùng dùng mô hình Logistic Regression với `class_weight="balanced"` để kết quả chỉ khác nhau ở cách xử lý giá trị thiếu.

Bảng 1.3: So sánh các cách xử lý giá trị thiếu (validation, lớp hủy)

Cách xử lý	Số dòng train	F1	PR-AUC	ROC-AUC
(a) Bỏ các dòng bị thiếu	122	0,184	0,289	0,524
(b) Bỏ cột `company`, điền median/mode	59.622	0,645	0,655	0,846
(c) Coi giá trị thiếu là một nhóm + cột cờ	59.622	0,647	0,654	0,849
(d) KNNImputer + nhóm riêng	59.622	0,647	0,654	0,849

Cách bỏ dòng gần như không dùng được: hầu hết các dòng đều thiếu `company` nên chỉ còn 122 dòng để huấn luyện, và PR-AUC chỉ bằng 0,289, sát với tỷ lệ hủy. Ba cách còn lại cho kết quả gần như nhau. Cách (c) được chọn vì không phải bỏ cột nào và giữ được ý nghĩa của ô trống là booking không qua đại lý hay công ty. KNN cho kết quả trùng với (c) vì trong các cột số chỉ có 4 ô bị thiếu.

1.5.4 Xử lý outlier

Ba cách xử lý các cột liên tục có đuôi dài được so sánh trên cùng mô hình: giữ nguyên, lấy `log1p`, và cắt tại phân vị 1% và 99% của tập train. `log1p` làm PR-AUC tăng từ 0,654 lên 0,674 và F1 từ 0,647 lên 0,667, còn cắt ngưỡng gần như không thay đổi gì (PR-AUC 0,654) vì chỉ 0,3–1% số dòng bị cắt. Do đó `log1p` được giữ lại. Cách này chủ yếu giúp mô hình tuyến tính, vì mô hình cây chỉ dựa vào thứ tự giá trị.

1.6 Mô hình và thiết lập thí nghiệm

Các mô hình được so sánh gồm một baseline luôn đoán lớp đa số (`DummyClassifier`), Logistic Regression, Decision Tree, Random Forest và HistGradientBoosting, cùng các phiên bản có `class_weight="balanced"`. Tất cả được huấn luyện trên tập train và so sánh trên tập validation. Họ mô hình tốt nhất sau đó được dùng để so sánh các cách xử lý mất cân bằng lớp và tinh chỉnh siêu tham số bằng `RandomizedSearchCV` (10 tổ hợp, cross-validation phân tầng 3 phần trên tập train, chấm theo average precision).

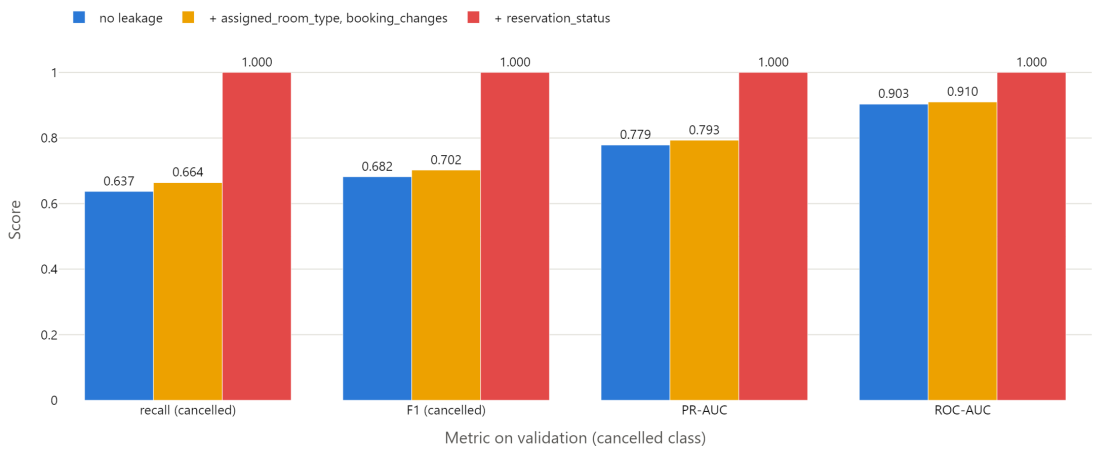
Bảng 1.4: So sánh các mô hình trên tập validation (lớp hủy)

Mô hình	Precision	Recall	F1	PR-AUC	Accuracy
Baseline (lớp đa số)	0,000	0,000	0,000	0,277	0,723
Logistic Regression	0,679	0,531	0,596	0,683	0,801
Logistic Regression (balanced)	0,562	0,822	0,667	0,674	0,773
Decision Tree	0,719	0,576	0,640	0,701	0,820
Random Forest	0,741	0,648	0,692	0,775	0,840
HistGradientBoosting	0,733	0,637	0,682	**0,779**	0,835
HistGradientBoosting (balanced)	0,602	0,856	0,707	0,775	0,803

Các mô hình dạng cây tốt hơn rõ rệt so với Logistic Regression (PR-AUC khoảng 0,78 so với 0,67–0,68), phù hợp với các quan hệ phi tuyến thấy ở phần EDA. HistGradientBoosting có PR-AUC cao nhất và huấn luyện nhanh hơn Random Forest nhiều, nên được dùng cho các bước tiếp theo. Phiên bản balanced gần như không làm thay đổi PR-AUC mà chủ yếu đổi giữa precision và recall.

1.6.1 Thí nghiệm rò rỉ dữ liệu

Để thấy vì sao các cột có sau khi đặt phòng phải bị loại, cùng một mô hình HistGradientBoosting được huấn luyện lại sau khi thêm các cột đó vào.



Hình 1.4: Thêm thông tin lúc check-in làm PR-AUC tăng từ 0,779 lên 0,793, còn thêm `reservation_status` cho kết quả tuyệt đối 1,000 nhưng không có ý nghĩa.

Khi có `reservation_status`, mọi chỉ số đều bằng 1,000 vì cột này chính là kết quả cần dự đoán. Trường hợp thêm `assigned_room_type` và `booking_changes` khó nhận ra hơn: PR-AUC chỉ tăng nhẹ, rất dễ bị hiểu nhầm là mô hình được cải thiện, nhưng tại thời điểm dự đoán thì hai cột này chưa tồn tại.

1.6.2 Xử lý mất cân bằng lớp

Bảng 1.5: Các cách xử lý mất cân bằng với HistGradientBoosting (validation, lớp hủy)

Cách xử lý	Precision	Recall	F1	PR-AUC
Không xử lý	0,733	0,637	0,682	0,779
`class_weight="balanced"`	0,602	0,856	0,707	0,775
SMOTE (chỉ trên tập train)	0,688	0,712	0,700	0,770
Điều chỉnh ngưỡng (0,374)	0,666	0,770	0,715	0,779

Cả ba cách đều làm tăng recall của lớp hủy. Điều chỉnh ngưỡng bằng cross-validation trên tập train cho F1 cao nhất và giữ nguyên PR-AUC, vì mô hình không đổi mà chỉ đổi ngưỡng phân loại. `class_weight` cho recall cao nhất nhưng precision giảm còn 0,602. SMOTE có PR-AUC thấp nhất, có thể do việc nội suy trên cả các cột one-hot tạo ra những booking không có thật, chẳng hạn một booking nửa thuộc City Hotel nửa thuộc Resort Hotel.

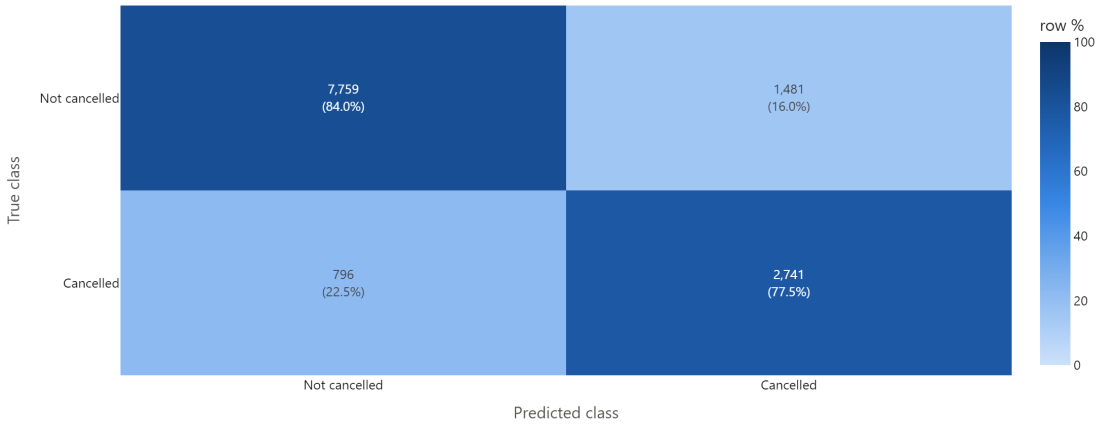
1.7 Kết quả và đánh giá

Mô hình cuối là HistGradientBoosting đã tinh chỉnh (learning rate 0,1, 200 vòng lặp, 31 lá, tối thiểu 100 mẫu mỗi lá, L2 regularization 1,0) với ngưỡng phân loại 0,364, được chọn lại bằng cross-validation trên tập train. Mô hình được huấn luyện lại trên train và validation (72.398 dòng) rồi đánh giá một lần trên tập test.

Bảng 1.6: Mô hình cuối trên tập validation và tập test (lớp hủy)

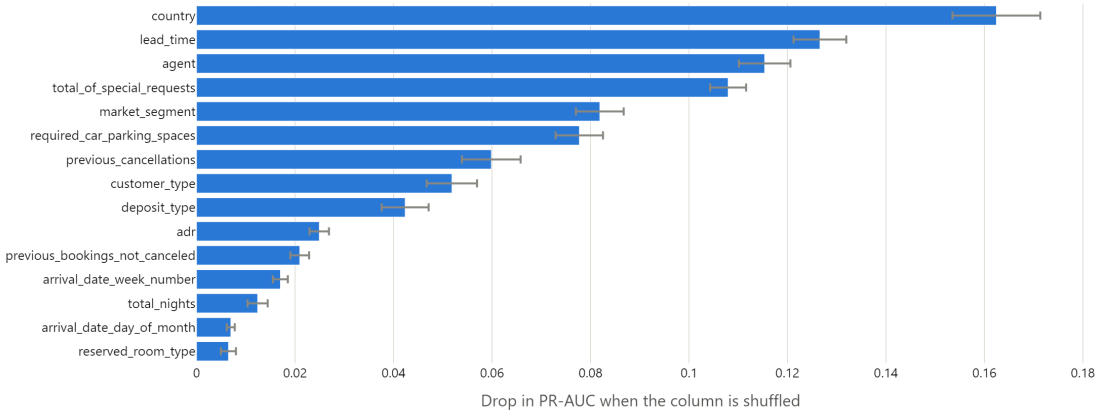
Mô hình	Tập	Precision	Recall	F1	PR-AUC	ROC-AUC	Accuracy
Baseline	test	0,000	0,000	0,000	0,277	0,500	0,723
HGB tinh chỉnh + ngưỡng	validation	0,660	0,782	0,715	0,783	0,905	0,828
HGB tinh chỉnh + ngưỡng	**test**	**0,649**	**0,775**	**0,707**	**0,775**	**0,900**	**0,822**

Kết quả trên tập test chỉ thấp hơn validation tối đa 0,011, nên mô hình không bị overfit lên tập validation (Bảng 1.6). So với baseline, PR-AUC tăng từ 0,277 lên 0,775 và F1 từ 0 lên 0,707, trong khi accuracy chỉ tăng từ 72,3% lên 82,2%. Trong 3.537 booking bị hủy ở tập test, mô hình phát hiện được 2.741 booking (77,5%); đổi lại có 1.481 trong 9.240 booking không hủy (16,0%) bị cảnh báo nhầm (Hình 1.5).



Hình 1.5: Trên tập test, mô hình phát hiện 77,5% số booking bị hủy và cảnh báo nhầm 16,0% số booking không hủy.

Permutation importance trên tập test phù hợp với phần EDA (Hình 1.6). Xáo trộn `country` làm PR-AUC giảm 0,162, tiếp theo là `lead_time` (0,127), `agent` (0,115), `total_of_special_requests` (0,108) và `market_segment` (0,082). `deposit_type` chỉ đứng thứ chín (0,042) vì sau khi bỏ trùng chỉ còn rất ít booking Non Refund.

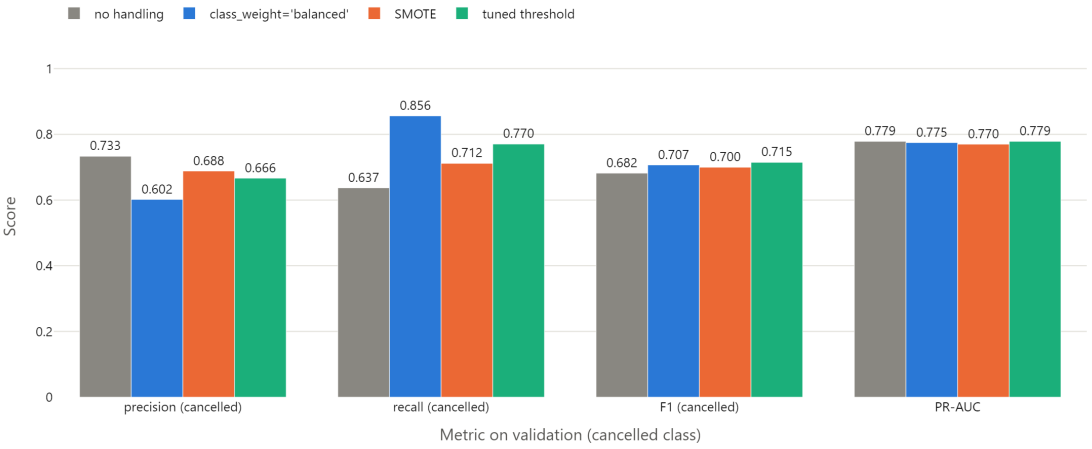


Hình 1.6: Quốc gia, thời gian đặt trước, đại lý và số yêu cầu đặc biệt là các đặc trưng quan trọng nhất, khớp với phần EDA.

Phân tích lỗi. 796 booking hủy bị bỏ sót có đặc điểm giống các booking ít khả năng hủy: trung vị thời gian đặt trước chỉ 35 ngày (so với 94 ngày ở các booking hủy được phát hiện) và 66,3% có yêu cầu đặc biệt (so với 32,0%). Lý do hủy của các booking này, như thay đổi kế hoạch hay ốm đau, không có trong dữ liệu đặt phòng, nên đây là giới hạn của dữ liệu hơn là của mô hình.

1.8 Hình trả lời câu hỏi

Câu hỏi: Xử lý mất cân bằng lớp có giúp phát hiện được nhiều booking bị hủy hơn không?



Hình 1.7: Xử lý mất cân bằng làm recall của lớp hủy tăng từ 0,637 lên 0,856 với class weight và lên 0,770 khi điều chỉnh ngưỡng; điều chỉnh ngưỡng cho F1 cao nhất (0,715) mà không làm giảm PR-AUC.

Có. Khi không xử lý, mô hình bỏ sót 36,3% số booking bị hủy trong tập validation. Với ngưỡng 0,374, mô hình chỉ bỏ sót 23,0% mà vẫn giữ precision 0,666. Với khách sạn, một cảnh báo nhầm thường chỉ tốn một cuộc gọi xác nhận, còn bỏ sót một booking hủy có thể khiến một phòng bị bỏ trống, nên đánh đổi một phần precision để có recall cao hơn là hợp lý.

1.9 Kết luận

1. **Dòng trùng ảnh hưởng lớn đến mọi con số.** 34.050 dòng trùng, phần lớn là booking đoàn Non Refund bị hủy, làm tỷ lệ hủy tăng từ 27,7% lên 37,0%. Nếu không bỏ trùng, nhóm đặt trước 181–365 ngày ở City Hotel hủy 62,7% thay vì 45,7%.

2. **Đặt càng sớm càng dễ hủy**: ở City Hotel, từ 10,5% (đặt trước tối đa 7 ngày) lên 45,7% (181–365 ngày).

3. **Khách có kế hoạch rõ ràng ít hủy hơn**: một yêu cầu đặc biệt làm tỷ lệ hủy của khách mới giảm từ 34,7% xuống 23,2%, khách quen hủy 7,7% so với 28,5%, và không booking nào có yêu cầu chỗ đỗ xe bị hủy.

4. **Các cột có sau khi đặt phòng cho kết quả giả**: `reservation_status` cho PR-AUC bằng 1,000, còn hai cột có lúc check-in âm thầm nâng PR-AUC từ 0,779 lên 0,793.

5. **Điều chỉnh ngưỡng là cách xử lý mất cân bằng hiệu quả nhất**: recall lớp hủy tăng từ 0,637 lên 0,770, F1 cao nhất (0,715) và PR-AUC không giảm như khi dùng SMOTE.

6. **Mô hình cuối** (HistGradientBoosting tinh chỉnh, ngưỡng 0,364) đạt PR-AUC 0,775, F1 0,707 và recall 0,775 trên tập test, phát hiện 77,5% số booking bị hủy.

1.10 Hạn chế và hướng phát triển

- **Dữ liệu:** chỉ gồm hai khách sạn ở Bồ Đào Nha với ngày đến từ 2015 đến 2017, nên kết quả chưa chắc áp dụng được cho khách sạn ở nơi khác hoặc giai đoạn sau COVID. Dataset không có mã booking nên không phân biệt được dòng nhập trùng với các phòng của cùng một đoàn; quyết định bỏ trùng làm tỷ lệ hủy giảm từ 37,0% xuống 27,7%.

- **Cột `country`:** có thể được cập nhật lúc check-in. Nếu đúng như vậy, booking bị hủy sẽ thường bị bỏ trống hoặc ghi giá trị mặc định ở cột này, và một phần mức độ quan trọng của nó đến từ đó. Dữ liệu hiện có không đủ để kiểm chứng.

- **Đánh giá:** dữ liệu được chia ngẫu nhiên chứ không theo thời gian, nên kết quả trên test có thể lạc quan hơn khi dự đoán các booking trong tương lai.

- **Hướng phát triển:** chia dữ liệu theo thời gian (huấn luyện trên 2015–2016, kiểm tra trên 2017), thử bỏ cột `country` để kiểm tra rò rỉ, chọn ngưỡng dựa trên chi phí thực tế của phòng trống và cuộc gọi xác nhận, và dùng SMOTENC thay cho SMOTE thông thường.

1.11 Tài liệu tham khảo

- N. António, A. de Almeida, L. Nunes, *Hotel booking demand datasets*, Data in Brief 22 (2019) 41–49. CC BY 4.0.

- R for Data Science Online Learning Community, *TidyTuesday 2020-02-11: Hotels*, https://github.com/rfordatascience/tidytuesday.

- F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research 12 (2011) 2825–2830.
- G. Lemaître, F. Nogueira, C. K. Aridas, *Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning*, Journal of Machine Learning Research 18 (2017) 1–5.